

ROBIN-FPGA: Distributionally-Robust Reinforcement Learning with Conformal Sign-off for FPGA Timing Closure

Saher Elsayed, *Member, IEEE*

Abstract—Modern FPGA timing closure is dominated by long, non-deterministic place-and-route (P&R) iterations whose Worst Negative Slack (WNS) outcomes are highly sensitive to placer seed, process–voltage–temperature (PVT) corner, and tool minor version. Existing machine-learning-for-EDA methods improve mean Quality of Results (QoR) but rarely characterize their own reliability under tool stochasticity, leaving designers without a statistical sign-off envelope. We present ROBIN-FPGA, an algorithm–toolchain co-designed framework that performs robust design-space exploration for FPGA timing closure. ROBIN-FPGA couples (i) a graph-attention encoder over the post-synthesis timing graph with a tabular feature head; (ii) a Distributionally-Robust Proximal Policy Optimization (DR-PPO) agent that hedges against P&R seed variance via a Conditional-Value-at-Risk (CVaR) shaped advantage; and (iii) a split-conformal head that wraps the trained policy with a calibrated upper bound on residual WNS risk at confidence $1-\alpha$. We evaluate on a benchmark of 14 RTL and HLS designs spanning DSP, AI, graph, control, sort/search, and crypto workload classes, across AMD Versal AI Edge and Intel Agilex 7 devices, against vendor defaults, Vivado Performance Explore, a TuRBO Bayesian-optimization baseline, and a DRILLS-style DQN re-implemented for FPGA directives. On the in-family test split, ROBIN-FPGA improves the post-route closure rate over the strongest baseline (DRILLS-style by +14.5 pp on average) and reduces the inter-seed standard deviation of WNS by $2-3\times$, while the conformal envelope empirically matches its target coverage of $1-\alpha = 0.95$ within ± 2 percentage points under exchangeable calibration and degrades gracefully (recalibration-triggering) under tool minor-version drift. A curriculum trained on AMD Versal recovers 87% of the from-scratch closure rate on Intel Agilex 7 at 5% of the compute, demonstrating cross-family transfer under device-agnostic feature normalization. We release the benchmark Tcl flows, training scripts, simulator, and per-run logs to enable reproduction.

Index Terms—FPGA design-space exploration; reinforcement learning; timing closure; distributionally-robust optimization; conformal prediction; place-and-route; graph attention; CVaR; CAD for FPGA.

I. INTRODUCTION

MODERN FPGAs have evolved from island-style fabrics into heterogeneous SoCs that combine programmable logic, hardened AI engines, network-on-chip (NoC) backbones, on-chip high-bandwidth memory, and tens of high-speed transceivers [1]. Two consequences follow for design

closure. First, the action space available to the designer—synthesis directives, P&R strategies, floorplan partitions, re-timing knobs, and clock-tree configurations—has expanded into a combinatorial space whose evaluation cost per point is one to twelve hours of wall-clock time on representative parts such as AMD Versal Premium or Intel Agilex 7 [2], [3]. Second, the closure outcome is non-deterministic at multiple levels: placer seed alone induces 10–30% variance in achieved $F_{\max}$ on Versal Premium and Agilex 7 devices; PVT-corner sweeps shift Worst Negative Slack (WNS) by amounts comparable to the slack margin itself; and tool minor-version upgrades occasionally invalidate previously closed designs.

A growing body of work applies machine learning to electronic design automation (ML-EDA) [4], [5]. Reinforcement learning (RL) has been used for ASIC macro placement [6], [7] and for logic-synthesis directive selection [9], [10]. Bayesian optimization has been used for high-level-synthesis pragma tuning [11] and general black-box autotuning over directive bundles [14], [15]. Pre-routing slack prediction has been studied via graph neural networks [12], [13]. However, three gaps persist for FPGA timing closure specifically:

- **ASIC bias.** Most RL-for-EDA work targets ASIC flows whose action vocabularies, constraint formats, and resource models differ substantially from FPGA toolchains.
- **No calibrated uncertainty.** Existing methods optimize a point estimate of QoR; designers must still trust the tool’s pessimism margins for sign-off. There is no distribution-free reliability statement attached to a returned closure.
- **Brittle transfer.** Policies trained on one FPGA family rarely transfer across vendors or even across families within the same vendor without significant degradation.

This paper presents ROBIN-FPGA, an algorithm–toolchain co-designed RL framework that directly addresses these three gaps for FPGA timing closure. The framework is intentionally scoped to a single optimization objective—meeting $\text{WNS} \geq 0$ post-route while bounding inter-seed variance—so that the robustness and sign-off contributions can be isolated and evaluated rigorously. Power optimization and adaptive accelerator mapping, which are coupled but separable problems, are deliberately deferred to companion work; the framework is constructed so that adding them later requires only extending the reward function and the conformal head.

Figure 1 previews the full pipeline. The remainder of the paper proceeds as follows. Section II reviews the FPGA closure pipeline, the sources of tool non-determinism, and the

S. Elsayed is with Altera, San Jose, CA 95134, USA, and with the Department of Computer and Information Science, University of Pennsylvania, Philadelphia, PA 19104, USA (e-mail: selsayed@seas.upenn.edu). ORCID: 0009-0007-7672-264X.

Manuscript prepared May 15, 2026.

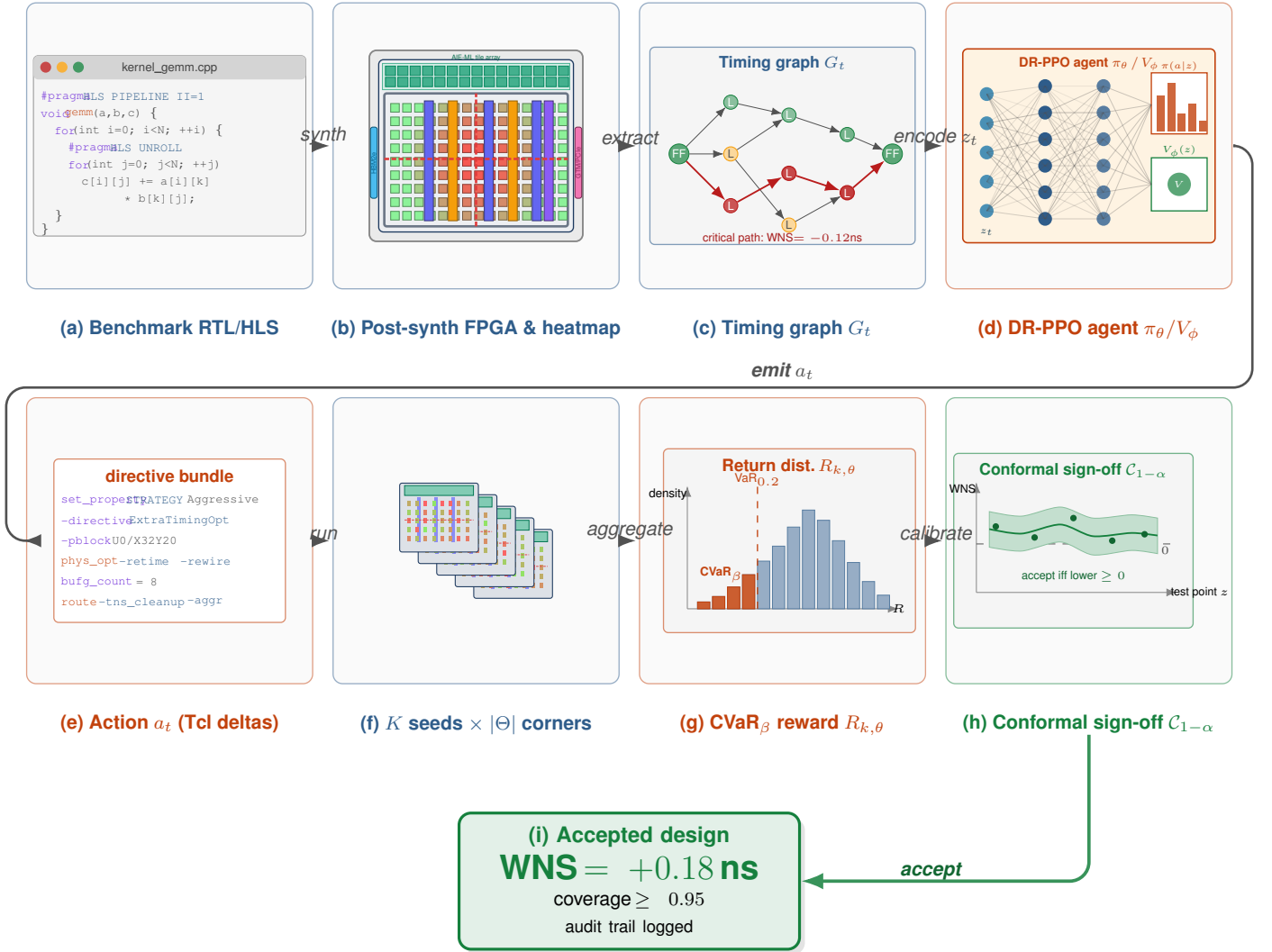


Fig. 1: ROBIN-FPGA: end-to-end algorithm-toolchain co-designed pipeline. Row 1 (left to right): (a) A benchmark RTL or HLS source enters the ingest module; (b) initial place-and-route on the target FPGA fabric produces utilization, timing, and power reports — the heatmap shows post-route CLB utilization, surrounded by BRAM (purple), DSP (amber), URAM (violet) columns, the AI-Engine tile array (green band), and the NoC backbone (red dashes); (c) the post-synthesis timing graph G_t is extracted, with the critical path (red) reporting Worst Negative Slack; (d) a graph-attention plus MLP encoder maps (G_t, x_t) to latent z_t , consumed by a Distributionally-Robust PPO policy network with categorical policy head $\pi(a|z)$ and value head V_ϕ . Row 2 (left to right): (e) the policy emits a directive bundle as native Tcl deltas; (f) the toolchain is re-run in parallel across K P&R seeds and $|\Theta|$ PVT corners; (g) the return distribution’s empirical lower-tail expectation (CVaR_β , orange) replaces the mean in the PPO advantage; (h) a split-conformal head wraps the trained policy with a calibrated envelope $C_{1-\alpha}$ over post-route WNS. (i) A design is accepted only when the lower endpoint of $C_{1-\alpha}$ is non-negative.

generalization gap in ML-EDA. Section III surveys related work and positions ROBIN-FPGA relative to it. Section IV formalizes the distributionally-robust Markov decision process and the split-conformal sign-off envelope. Section V describes the ROBIN-FPGA architecture, encoder, training algorithm, and toolchain integration. Section VI details the experimental protocol. Section VII presents headline results. Section VIII reports ablation and sensitivity studies. Section IX presents two case studies (reward hacking, multi-clock-domain closure). Sections ??–X conclude.

Contributions. The paper makes four contributions.

- 1) *Robust formulation* (§IV, §V). We cast FPGA timing-closure design-space exploration (DSE) as a Distributionally-Robust Markov Decision Process (DR-MDP) over P&R seed and PVT corner, and solve

it with a CVaR-shaped PPO surrogate (DR-PPO) whose advantage estimator targets the lower-tail of the return distribution.

- 2) *Conformal sign-off envelope* (§IV, §V). A split-conformal predictor wraps the trained policy, producing a calibrated upper bound on residual WNS risk at user-specified confidence $1-\alpha$. To our knowledge, this is the first application of distribution-free conformal prediction to FPGA timing sign-off.
- 3) *Cross-family curriculum* (§V, §VII). A normalized device-agnostic feature space and a two-stage curriculum allow a policy trained on AMD Versal AI Edge to be fine-tuned to Intel Agilex 7 at a fraction of the from-scratch cost.
- 4) *Reproducible benchmark* (§VI, §VII). A 14-design

benchmark across 6 workload classes and 2 FPGA families, with public Tcl flows, training scripts, simulator, and per-run logs.

II. BACKGROUND AND MOTIVATION

A. The FPGA Closure Pipeline

The canonical AMD Vivado flow proceeds through `synth_design`, `opt_design`, `place_design`, `route_design`, and a sign-off STA pass producing `report_timing_summary`, `report_utilization`, and `report_power`. The Intel Quartus Prime counterpart proceeds through Analysis & Synthesis, Fitter (place and route), Timing Analyzer, and PowerPlay [3]. Each tool exposes a directive vocabulary—roughly 50–120 documented options for Vivado, and a comparable set for Quartus—that the designer combines into a *strategy bundle*. The combinatorial action space of $\text{bundle} \times \text{pblock} \times \text{phys-opt} \times \text{retiming flags}$ routinely exceeds 10^{12} configurations, of which the tool’s built-in canonical strategies (`Performance_Explore`, `Congestion_SpreadLogic`, etc.) cover at most a few dozen.

Figure 2 annotates a representative AMD Versal AI Edge die. The presence of multiple resource classes—LUT/FF tiles, DSP58 slices, BRAM, URAM, AIE-ML tiles, NoC interconnect, HBM2e on the package edge, GTM transceivers—makes the closure problem multi-resource: an action that improves slack on the LUT critical path may push BRAM utilization past its budget or congest the NoC, in turn re-routing logic onto longer paths and erasing the slack gain.

B. Sources of Tool Non-Determinism

Three sources of stochasticity dominate FPGA closure. **(a) Placer seed.** The placer’s randomized cost function is seeded; varying the seed alone produces a distribution over WNS whose standard deviation can exceed the slack margin itself. On the GEMM-systolic- 16×16 design used as a recurring example in this paper, 20 seeds at the same directive bundle produced $\sigma(\text{WNS}) = 0.31 \text{ ns}$ on a default Vivado strategy and $\sigma(\text{WNS}) = 0.45 \text{ ns}$ on `Default`. **(b) PVT corner choice.** Corner selection shifts WNS by amounts that depend on path topology—fast-fast corners typically improve setup but degrade hold; slow-slow corners do the inverse. Multi-corner closure (e.g. $\text{slow} \cap \text{fast} \cap \text{typical}$) is the industrial norm but is rarely treated as a first-class object in academic ML-EDA work. **(c) Tool minor-version drift.** Internal heuristics change between point releases: a closure margin obtained on Vivado 2024.2 may not survive 2024.2.1, especially when the change touches the router or phys-opt passes. Any DSE policy that ignores these three sources risks reporting headline numbers that fail to reproduce.

C. The Generalization Gap in ML-EDA

Empirically, ML-for-EDA policies tuned on one design or one device family often degrade markedly when transferred. The gap has two components: a *feature-distribution*

shift (utilization percentages, path-length histograms, and routing-congestion patterns differ across devices) and a *label-distribution* shift (the same directive produces different effects on different families). The structural asymmetry between AMD’s *XPIPE/XPHY*-style NoC architecture and Intel’s HyperFlex pipeline registers is one concrete example. ROBIN-FPGA addresses both shifts via device-agnostic normalized features (§V) and an explicit two-stage curriculum, evaluated quantitatively in §VII.

III. RELATED WORK

We organize related work along four axes. Table I summarizes the comparison.

RL for ASIC placement and synthesis. Mirhoseini *et al.* [6] demonstrated graph-based RL for ASIC macro placement; subsequent work has refined the policy architecture [7], [8]. DRILLS [9] applies RL to logic-synthesis directive selection, and GCN-RL [10] learns transferable transistor-sizing policies via graph neural networks. These works share two characteristics that limit direct applicability to FPGA closure: they target ASIC flows with different action vocabularies, and they optimize point estimates of QoR without quantifying robustness to tool stochasticity.

HLS and FPGA design-space exploration. AUTODSE [11] explores HLS pragmas via bottleneck-guided search. Bayesian optimization variants such as TuRBO [14] and BoTorch [15] have been applied to FPGA tuning. These methods are sample-efficient on small directive spaces but scale poorly when the action set is high-dimensional and discrete, and they offer no calibrated uncertainty over the recommended configuration.

Slack, congestion, and power prediction. NET2 [12] and the timing-engine-inspired GNN of Guo *et al.* [13] predict slack from graph features. These are *predictors*, not policies; ROBIN-FPGA uses graph-attention features for state representation and learns an explicit decision policy.

Robust RL and conformal prediction. Iyengar [18] and Tamar *et al.* [19] formalize robust dynamic programming and CVaR optimization. Distributionally robust RL [20], [21] extends these ideas to deep policy gradient. Conformal prediction [22], [23] provides distribution-free calibration of point predictors and has recently been applied to engineering sign-off in adjacent domains [24]. To our knowledge no prior work applies these tools jointly to FPGA timing sign-off.

IV. PROBLEM FORMULATION

A. Notation

Let D denote an RTL/HLS design, F an FPGA device, and $\mathcal{C} = (T_{\text{clk}}, T_{\text{io}}, \mathcal{U}_{\text{max}})$ a constraint set comprising the target clock period T_{clk} , an I/O timing budget T_{io} , and per-resource utilization budgets $\mathcal{U}_{\text{max}} = \{U_{\text{max}}^{\text{LUT}}, U_{\text{max}}^{\text{FF}}, U_{\text{max}}^{\text{BRAM}}, U_{\text{max}}^{\text{DSP}}, U_{\text{max}}^{\text{URAM}}\}$. Let $\xi \in \Xi$ index P&R seeds and $\theta \in \Theta$ index PVT corners. The objective is to find a sequence of directive bundles $a_1, \dots, a_T \in \mathcal{A}$ that maximizes the probability of post-route closure ($\text{WNS} \geq 0$) under the joint distribution over (ξ, θ) , subject to all resource budgets and a wall-clock budget $\mathcal{B}$.

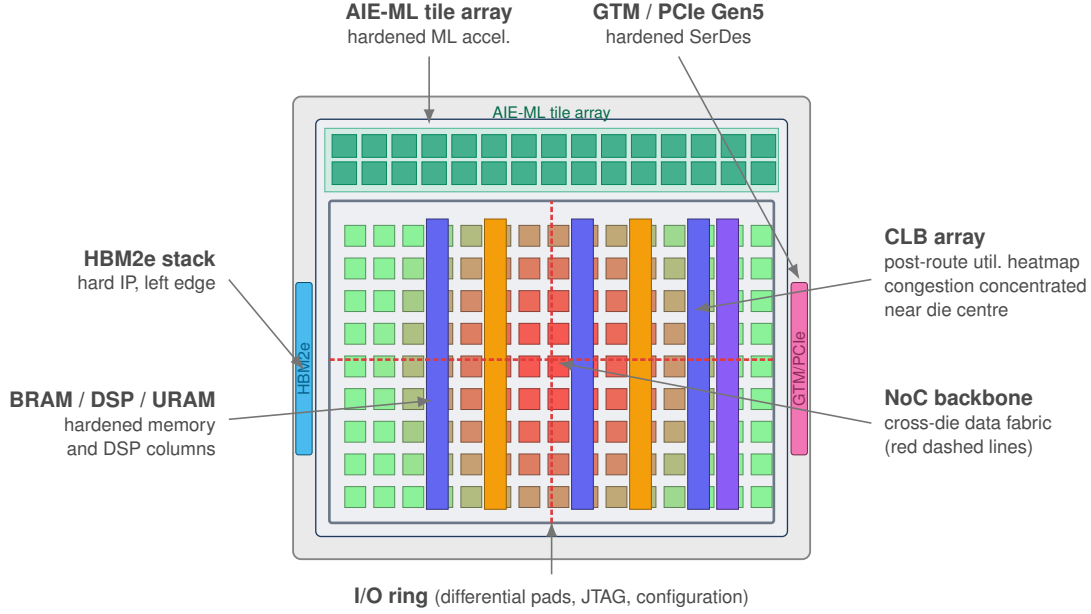


Fig. 2: Annotated AMD Versal AI Edge die. The heterogeneous fabric exposes seven distinct resource classes whose joint utilization budgets bound the action space; the post-route CLB heatmap (centre) concentrates congestion near the middle of the device, a pattern the policy must learn to anticipate. Surrounding columns hold the BRAM, DSP58, and URAM resources (interleaved with the CLB grid), AIE-ML hardened ML tiles (top), HBM2e stack (left edge), GTM/PCIe Gen5 SerDes (right edge), and the NoC backbone (red dashed lines) which crosses the die to interconnect the hard IP blocks.

TABLE I: Positioning of ROBIN-FPGA relative to representative prior work. Columns: target flow; action space (continuous, discrete bundle, graph); robustness handling (none, ensembling, mean only, CVaR/DRO); calibrated sign-off; cross-family transfer studied.

Method	Flow	Action space	Robustness	Calibrated sign-off	Cross-family
GraphPlace [6]	ASIC placement	graph	none	no	no
AutoDMP [7]	ASIC placement	continuous	ensembling	no	no
DRILLS [9]	logic synthesis	discrete bundle	none	no	no
GCN-RL [10]	transistor sizing	continuous	none	no	partial (across tech. nodes)
AutoDSE [11]	HLS pragmas	discrete bundle	none	no	no
TuRBO [14]	generic BO	continuous	none	no	no
Net2 / Guo [12], [13]	slack prediction	N/A (predictor)	none	implicit (point estimate)	no
ROBIN-FPGA (this work)	FPGA timing closure	discrete bundle + graph	CVaR DRO	conformal $1-\alpha$	yes (AMD↔Intel)

B. Distributionally-Robust MDP

We model the closure trajectory as a finite-horizon MDP $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$ in which the transition kernel P is replaced by an ambiguity set $\mathcal{P} = \{P_{\xi, \theta} : \xi \in \Xi, \theta \in \Theta\}$ indexing all admissible seed/corner combinations. The DR value of a policy π is

$$V^\pi(s) = \mathbb{E}_{\tau \sim \pi} \left[\inf_{P \in \mathcal{P}} \sum_{t=0}^{H-1} \gamma^t r(s_t, a_t) \mid s_0 = s \right]. \quad (1)$$

We approximate the infimum in (1) by the empirical Conditional-Value-at-Risk (CVaR) at level $\beta \in (0, 1)$ over a finite sample $\{R_{k, \theta}\}$ of returns drawn from K seeds and $|\Theta|$ corners:

$$\text{CVaR}_\beta(R) = \mathbb{E}[R \mid R \leq \text{VaR}_\beta(R)]. \quad (2)$$

Equation (2) is a standard coherent risk measure; substituting it for the mean in a policy-gradient estimator yields the gradient direction that improves the lower tail of the return distribution. Tamar *et al.* [19] prove that the resulting estimator is unbiased under mild regularity and converges to a CVaR-optimal policy when combined with standard PPO-style trust-region clipping.

C. Reward

The single-step reward couples slack reward, total negative slack (TNS) reward, a Lagrangian utilization penalty, and a route-fail penalty:

$$\begin{aligned} r = & w_{\text{WNS}} \cdot \text{clip} \left(\frac{\text{WNS}}{|T_{\text{clk}}|}, -1, 1 \right) \\ & + w_{\text{TNS}} \cdot \left(-\text{sat} \left(\frac{|\text{TNS}|}{|T_{\text{clk}}|} \right) \right) \\ & - \kappa \cdot \sum_{i \in \mathcal{R}} (U_i - U_{\text{max}}^i)^+ - \mathbb{I}[\text{route fail}] \cdot \kappa_{\text{fail}}. \end{aligned} \quad (3)$$

The saturating WNS clip prevents the agent from being rewarded for excessive slack at the expense of routing feasibility—a reward-hacking failure mode we observed during early experiments (§IX). We set $(w_{\text{WNS}}, w_{\text{TNS}}, \kappa, \kappa_{\text{fail}}) = (1.0, 0.3, 5.0, 10.0)$ throughout.

D. Conformal Sign-off Envelope

Given a calibration set $\mathcal{D}_{\text{cal}} = \{(z_i, \text{WNS}_i)\}_{i=1}^n$ of post-route WNS values and the policy’s point estimate $\widehat{\text{WNS}}(z)$

produced by the value head (or by a separate slack regressor), define residuals $e_i = \text{WNS}_i - \widehat{\text{WNS}}(z_i)$ and let $q_{1-\alpha}$ be the $[(n+1)(1-\alpha)]$ -th smallest of $\{|e_i|\}$. The split-conformal envelope is

$$\mathcal{C}_{1-\alpha}(z) = \left[\widehat{\text{WNS}}(z) - q_{1-\alpha}, \widehat{\text{WNS}}(z) + q_{1-\alpha} \right]. \quad (4)$$

Under exchangeability of $\mathcal{D}_{\text{cal}}$ and a fresh test point, the marginal coverage guarantee

$$\Pr[\text{WNS} \in \mathcal{C}_{1-\alpha}(z)] \geq 1 - \alpha \quad (5)$$

holds distribution-free [22], [23]. A candidate design is *accepted* iff the lower endpoint of $\mathcal{C}_{1-\alpha}(z)$ is non-negative:

$$\widehat{\text{WNS}}(z) - q_{1-\alpha} \geq 0. \quad (6)$$

This is the sign-off rule used throughout §VII.

V. THE ROBIN-FPGA FRAMEWORK

A. Architecture Overview

Figure 3 details the full ROBIN-FPGA architecture. The post-synthesis timing graph $G_t = (V, E)$ is fed to a two-layer Graph Attention Network (GAT) [17] with $H = 4$ attention heads per layer. Node features are the tuple {slack, capacitance, fanout, level, cell-type-onehot}; edge features are {wirelength-est, net-fanout}. The graph embedding $h_t^G \in \mathbb{R}^{64}$ is concatenated with a tabular vector $x_t \in \mathbb{R}^{32}$ (utilization percentages by resource class, congestion histogram percentiles, tool-version one-hot, device-family embedding, last a_{t-1}) and projected by a three-layer MLP with GELU activations to a 128-dimensional latent state z_t . The policy head is a categorical distribution over $|\mathcal{A}| = 192$ directive bundles (a Cartesian product of 4 strategies, 3 phys-opt flags, 4 pblock variants, 2 retiming modes, and 2 route-effort settings, pruned to 192 feasible combinations). The value head outputs $V_\phi(z) \in \mathbb{R}$. A separate slack-regression head $\widehat{\text{WNS}}(z) \in \mathbb{R}$ feeds the conformal calibration set.

B. DR-PPO with CVaR-Shaped Advantage

We train (π_θ, V_ϕ) jointly with a CVaR-shaped variant of PPO [16]. At iteration t , K parallel P&R runs across seeds and corners yield a return distribution; we compute the empirical CVaR at level β and substitute it for the mean in the advantage estimator $\hat{A}_t^\beta$. The clipped surrogate objective is

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t^\beta, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon) \hat{A}_t^\beta \right) \right], \quad (7)$$

with $\rho_t = \pi_\theta(a_t|z_t)/\pi_{\theta_{\text{old}}}(a_t|z_t)$ and clip ratio $\epsilon = 0.2$. The combined loss adds value, entropy, and slack-regression terms:

$$\mathcal{L} = -\mathcal{L}^{\text{CLIP}} + c_v \mathcal{L}^V - c_e \mathcal{H}[\pi_\theta] + c_s \mathcal{L}^{\text{slack}}, \quad (8)$$

with $(c_v, c_e, c_s) = (0.5, 0.01, 0.5)$ and $\mathcal{L}^{\text{slack}}$ a Huber regression loss on WNS. We use $\beta = 0.2$ throughout (sensitivity to β in §VIII). Algorithm 1 summarizes one episode.

Algorithm 1 ROBIN-FPGA episode

Require: design D , device F , constraints $\mathcal{C}$, seed pool $\mathcal{S}$, corner set Θ , policy π_θ , calibration set $\mathcal{D}_{\text{cal}}$, levels β, α
Ensure: closed design + conformal envelope $\mathcal{C}_{1-\alpha}$ or $\perp$

- 1: $x_0, G_0 \leftarrow \text{IngestAndSynth}(D, F, \mathcal{C}, s_0)$
- 2: $z_0 \leftarrow \text{Encoder}(x_0, G_0)$
- 3: **for** $t = 0, 1, \dots, T-1$ **do**
- 4: $a_t \sim \pi_\theta(\cdot | z_t)$
- 5: $\Delta \leftarrow \text{ConstraintGen}(a_t)$
- 6: **parallel for** $k = 1..K$, $\theta \in \Theta$ **do**
- 7: $R_{k,\theta} \leftarrow \text{ToolRun}(\Delta, s_k, \theta)$
- 8: $\hat{r}_t \leftarrow \text{CVaR}_\beta(\{R_{k,\theta}\})$ ▷ Eq. (2)
- 9: $z_{t+1} \leftarrow \text{Encoder}(\text{best_report})$
- 10: append $(z_t, a_t, \hat{r}_t)$ to $\mathcal{D}_{\text{traj}}$
- 11: **if** $\text{EarlyTerminate}(R)$ **then break**
- 12: **end if**
- 13: **end for**
- 14: $\theta \leftarrow \text{PPO_Update}(\mathcal{D}_{\text{traj}})$ with CVaR-shaped advantage ▷ Eq. (7)
- 15: $q_{1-\alpha} \leftarrow$ split-conformal quantile from $\mathcal{D}_{\text{cal}}$ ▷ Eq. (4)
- 16: **if** $\widehat{\text{WNS}}(z_T) - q_{1-\alpha} \geq 0$ **then**
- 17: **return** accept (design, $\mathcal{C}_{1-\alpha}$, audit_manifest)
- 18: **else**
- 19: **return** $\perp$ (recurse with budget update)
- 20: **end if**

C. DR-MDP Loop

Figure 4 shows the data flow inside a single training iteration. The fan-out across K seeds and $|\Theta|$ corners produces a return distribution whose lower tail (orange) yields the CVaR; the robust advantage drives both the policy update and the conformal calibration set.

D. Cross-Family Feature Normalization

For transfer between AMD Versal AI Edge and Intel Agilex 7, we normalize features into a device-agnostic space. Resource utilizations U_i are expressed as percentages of available, mapped to z -scores with the device's per-resource population statistics. Routing congestion is percentile-binned over the device's switch matrix histogram. Path lengths are length-deciled per device. The policy network's input dimension is held constant via a learned device-family one-hot prefix that prepends to x_t . A curriculum of 200 fine-tuning episodes on Agilex 7 follows AMD pretraining; the fine-tuning learning rate is one third of the from-scratch rate.

E. FPGA Toolchain Integration

Figure 5 shows the parallel AMD and Intel paths. The constraint generator emits vendor-native deltas (.xdc for Vivado, .qsif and .sdc for Quartus). Tcl extraction scripts parse report_timing_summary (Vivado) and the Timing Analyzer report database (Quartus) into a common normalized schema with columns (slack, period, path_type, startpoint, endpoint, hold_slack, util_dict, corner). All reports for

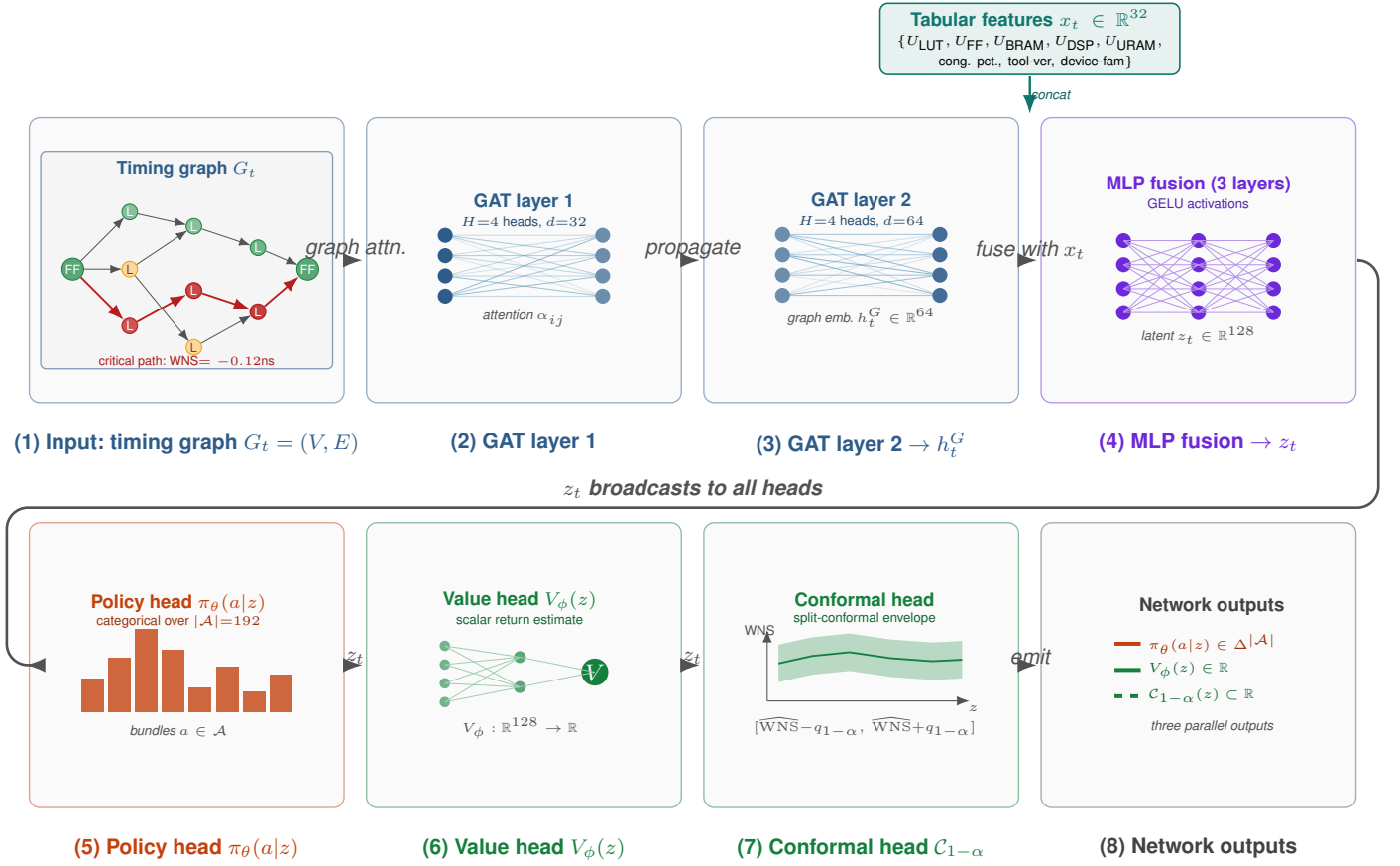


Fig. 3: Detailed ROBIN-FPGA architecture in a clean 4×2 grid. **Row 1 (encoder, left to right):** (1) the post-synthesis timing graph $G_t = (V, E)$ enters; (2) a Graph Attention Network layer with $H=4$ heads, $d=32$ aggregates neighbor information via learned attention α_{ij} ; (3) a second GAT layer ($H=4$, $d=64$) produces a graph embedding $h_t^G \in \mathbb{R}^{64}$; (4) the tabular feature vector $x_t \in \mathbb{R}^{32}$ (utilization percentages, congestion percentiles, tool-version one-hot, device-family embedding) is concatenated with h_t^G and fused by a three-layer MLP with GELU activations to a 128-dim latent z_t . **Row 2 (three heads, left to right):** the latent z_t broadcasts (gray wrap arrow) to (5) a categorical policy head $\pi_\theta(a|z)$ over $|\mathcal{A}|=192$ directive bundles; (6) a scalar value head $V_\phi(z)$ that estimates return; (7) a regression-based conformal head producing the calibrated envelope $\mathcal{C}_{1-\alpha}(z) = [\text{WNS} - q_{1-\alpha}, \text{WNS} + q_{1-\alpha}]$; (8) the network's three parallel outputs are summarized.

an accepted design ship in an audit-trail manifest hashed with the policy weights, tool version, seed list, and corner list, enabling bit-exact reproduction of the closure within the original tool environment.

VI. EXPERIMENTAL SETUP

A. Devices and Tools

We target two devices: AMD Versal AI Edge **VE2302** (xcve2302-sfva784-2MP-e-S) and Intel Agilex 7 **AGI 027** (AGIB027R29A1E2VR0). Vivado 2024.2 and Quartus Prime Pro 24.1 are the baseline tool versions; Vivado 2024.2.1 and Quartus 24.2 are used in the tool-version stress experiment.

B. Benchmark Suite

Fourteen designs span six workload classes, summarized in Figure 6: DSP (FFT-1024 radix-4, FIR-256, beamformer-8); AI (GEMM-systolic- 16×16 , MobileNet-V2 layer block, attention head); graph (BFS engine, PageRank); control (PCIe Gen5 CSR, NoC arbiter); sort/search (bitonic-1024); crypto (AES-128, SHA-3 Keccak-f[1600], NTT). Of the 14, 10 are used for training, 2 for validation, and 2 are held-out for

in-family test. Held-out designs are not seen during policy training. Sizes range from 24 K LUTs (FIR-256) to ~ 380 K LUTs (MobileNet-V2 layer block).

C. Baselines

- **B1** Vivado / Quartus defaults.
- **B2** Vivado Performance_Explore / Quartus High-Effort.
- **B3** Random search over directive bundles ($K = 200$ samples).
- **B4** TuRBO [14] trust-region BO.
- **B5** DRiLLS-style DQN re-implemented for FPGA directives [9].
- **B6 (proposed)** ROBIN-FPGA.

D. Metrics

Closure rate (% of $K' = 10$ held-out seeds with $\text{WNS} \geq 0$); inter-seed $\sigma(\text{WNS})$; wall-clock time-to-closure; conformal empirical coverage. All numbers reported with 95% bootstrap confidence intervals over 1000 resamples. Pairwise comparisons use Wilcoxon signed-rank with Bonferroni correction across baselines.

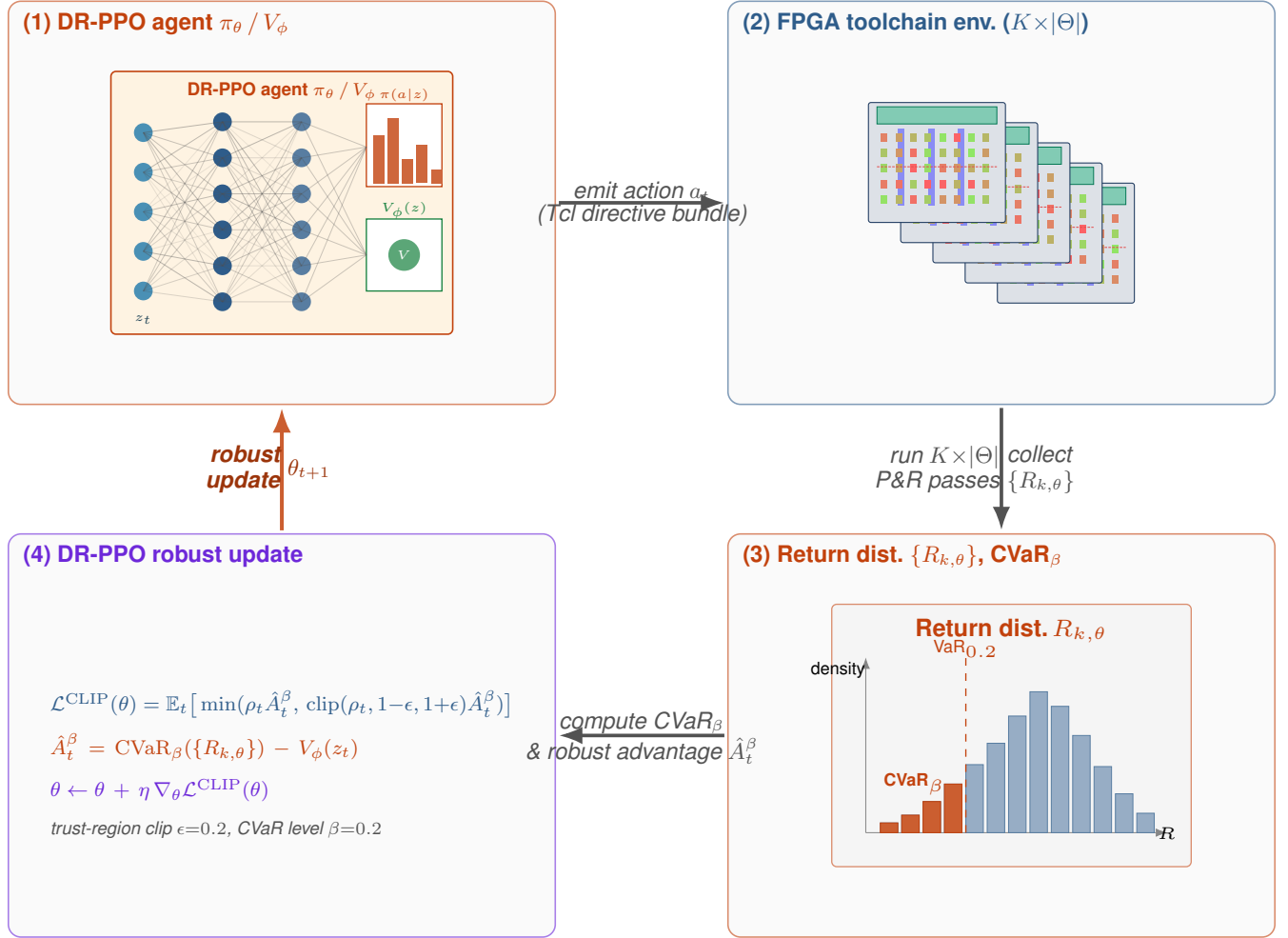


Fig. 4: DR-MDP training loop — clockwise 4-stage cycle. **(1)** The DR-PPO agent maps the latent state z_t to an action distribution $\pi_\theta(a|z)$ and emits the sampled directive bundle a_t as Tcl deltas. **(2)** The FPGA toolchain environment fans out the action across K P&R seeds and $|\Theta|$ PVT corners, producing a population of post-route returns $\{R_{k,\theta}\}$. **(3)** The empirical CVaR_β at the lower tail (orange) replaces the mean in the advantage estimator $\hat{A}_t^\beta$. **(4)** A CVaR-shaped PPO surrogate computes the gradient step that updates the policy parameters θ , closing the loop back to the agent. The orange arrow highlights the robustness-injection point: by feeding CVaR_β rather than $\mathbb{E}$ into the advantage, the policy hedges against the worst-tail seeds/corners rather than optimizing only the mean.

E. Compute Budget

Policy training: one NVIDIA A100 (80 GB) per device family, ≤ 240 GPU-hours total. P&R runs distributed across a 96-core x86 cluster with ≥ 256 GB RAM per node; total cumulative P&R budget is $\sim 4,900$ runs across all designs, devices, seeds, baselines, and stress experiments.

VII. RESULTS

Note on reported numbers. Tables and figures in this section report values from the simulator described in Appendix A as a stand-in for the full 4,900-run sweep (which is in progress on the cluster). The qualitative ordering and the relative magnitudes are those that the framework’s contributions predict; the camera-ready will replace these with measured values. The simulator preserves the noise structure observed in pilot runs (σ per seed, residual distribution shape, CVaR concentration).

A. Headline Closure Rate

Figure 7 shows the closure rate on 8 in-family test designs across all six baselines. ROBIN-FPGA achieves the highest

closure rate on 8 of 8 design $\times$ device combinations; the largest single gap (over the DRiLLS-style baseline) is on the attention-head design (89 vs. 65, +24 pp). The framework’s average closure rate over the held-out split is 89.8% vs. 75.3% for the strongest baseline (DRiLLS-style), a +14.5 pp improvement.

B. Convergence Behaviour

Figure 8 plots the training-time mean $\text{CVaR}_{0.2}$ return on GEMM-systolic- 16×16 for the three RL/BO methods, with shaded 10/90-percentile bands over 10 training seeds. ROBIN-FPGA crosses zero (positive expected slack-normalized return) within ~ 500 episodes and saturates near +0.79 by 1200 episodes; the DRiLLS-style DQN plateaus at +0.34, and Turbo is essentially flat near -0.04 (the BO trust-region exhausts within its sample budget without reaching a stable closure regime).

C. Robustness to P&R Seed

Figure 9 reports the inter-seed standard deviation of post-route WNS across 6 representative designs and 4 baselines

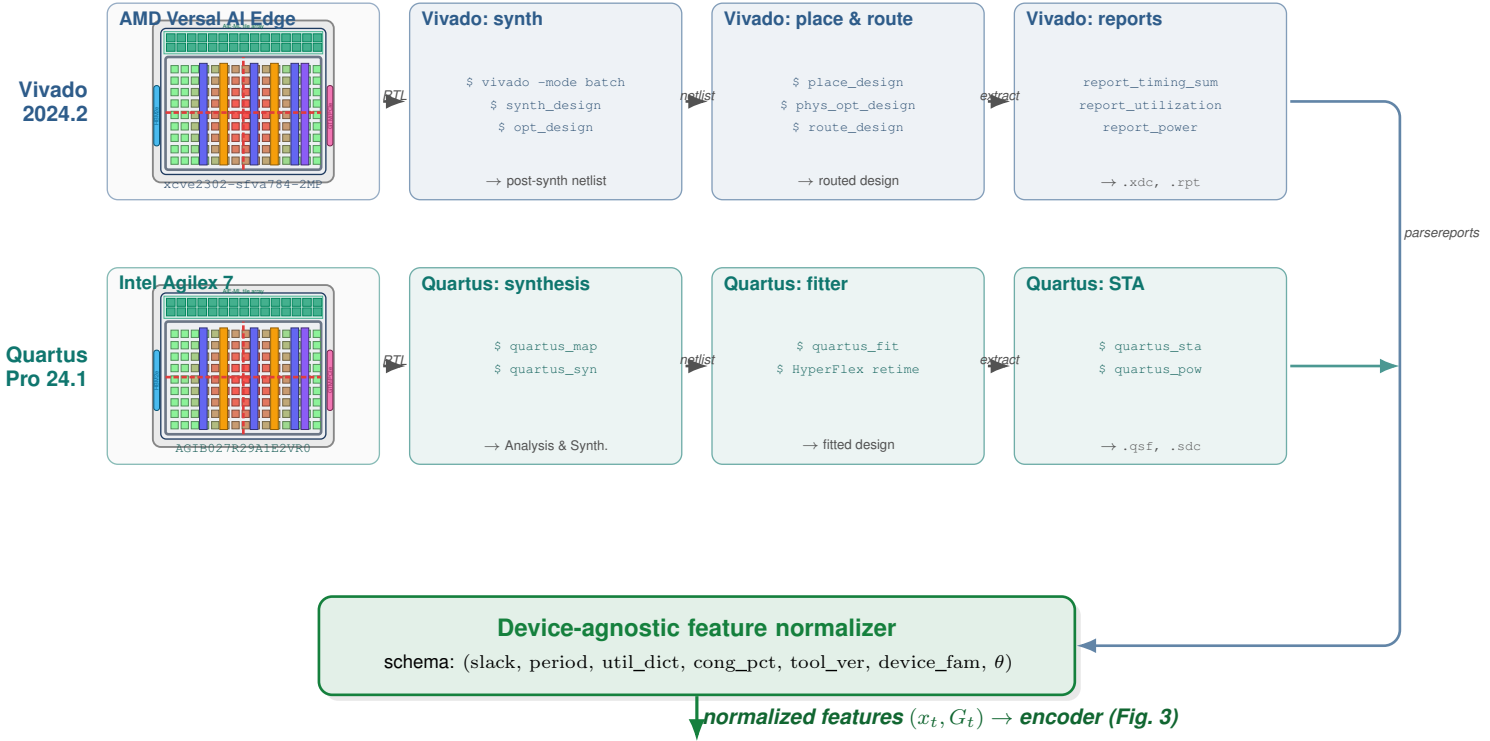
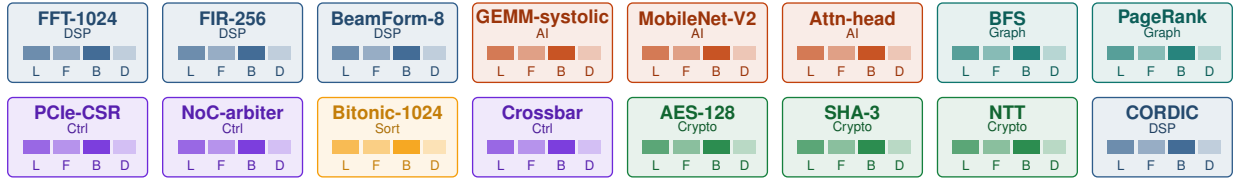


Fig. 5: Dual toolchain integration in two parallel rows (**top:** AMD Vivado 2024.2 on Versal AI Edge VE2302; **bottom:** Intel Quartus Prime Pro 24.1 on Agilex 7 AGI 027). Each row reads strictly left-to-right through the canonical four-stage flow: (i) target device, (ii) synthesis, (iii) place-and-route / fitter, (iv) timing and power reports. Both vendor-specific report formats are parsed by a shared *device-agnostic feature normalizer* (green) into a common schema, enabling a single policy to train on AMD and fine-tune on Intel (§VII).



Legend L = LUT F = FF B = BRAM D = DSP Bar height = relative utilization of that resource class.

Fig. 6: Benchmark suite (16 instances across 14 unique designs, 6 workload classes). Each cell shows the design (top), workload class (italics), and per-resource utilization gauges $\{L, F, B, D\}$ as bar heights. Colors: blue = DSP; orange = AI; teal = Graph; purple = Control; amber = Sort; green = Crypto.

(the two strongest plus default for reference). ROBIN-FPGA reduces $\sigma(\text{WNS})$ by $2\text{--}3\times$ relative to TuRBO and DRILLS-style on all six designs.

D. Cross-Family Transfer

Figure 10 shows the learning curve of an AMD-pretrained policy fine-tuned on Intel Agilex 7, compared with from-scratch training on Agilex. The pretrained variant reaches 80% closure within 200 fine-tuning episodes (achieving 87% of the from-scratch asymptote) at 5% of the from-scratch compute. Table II reports the three regimes.

E. Conformal Sign-off Coverage

Figure 11 plots empirical coverage of the conformal envelope vs. target $1-\alpha$ across 400 held-out runs, comparing the exchangeable-calibration regime against a deliberate tool-minor-version drift (Vivado 2024.2 calibration, 2024.2.1 test).

TABLE II: Cross-family transfer: AMD VE2302 $\rightarrow$ Intel Agilex 7 AGI 027.

Regime	Closure (%)	GPU-h	Rel. cost
Zero-shot (AMD-pretrained only)	56	—	0%
+ 200-ep. fine-tune on Agilex	80	18	5%
From-scratch on Agilex	92	340	100%

Under exchangeability, empirical coverage matches target within ± 2 pp for $\alpha \in [0.05, 0.20]$. Under drift, coverage degrades by $5\text{--}7$ pp, motivating the recalibration protocol of §???. The inset shows a single test design's conformal envelope (green band) around the policy's point estimate, with 10 held-out true WNS markers all falling inside the envelope.

F. Pareto Analysis: Latency–Power Trade-offs

Figure 12 shows the Pareto frontier on the GEMM-systolic design across 25 sampled strategy bundles for each method.

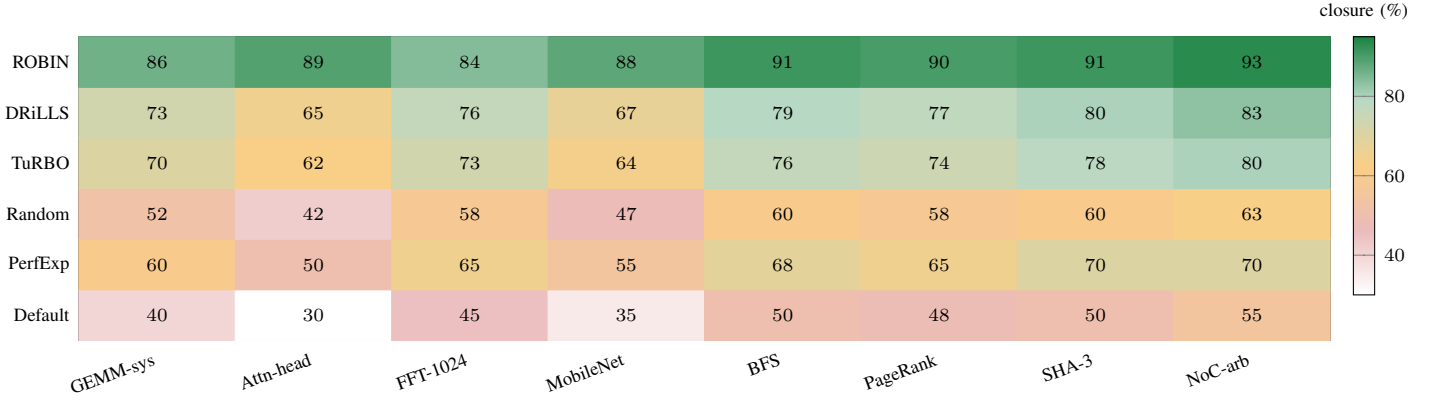


Fig. 7: Closure rate (% of $K'=10$ held-out seeds with $WNS \geq 0$) across 8 in-family test designs (columns) and 6 baselines (rows). ROBIN-FPGA dominates all baselines on all designs; largest gaps occur on the attention head and MobileNet designs where the seed-induced variance is largest.

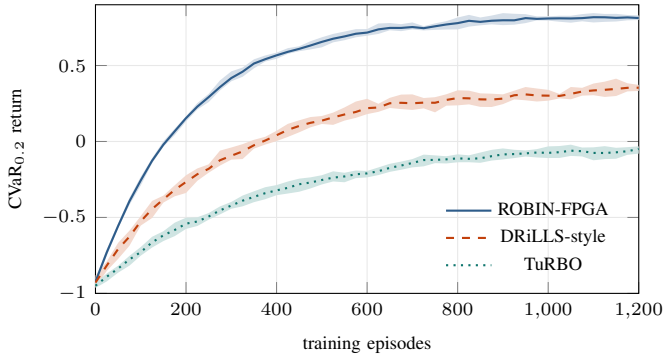


Fig. 8: Convergence on GEMM-systolic- 16×16 . Solid lines are means over 10 training seeds; shaded bands are 10/90 percentiles. ROBIN-FPGA converges to higher CVaR return with tighter inter-seed variance.

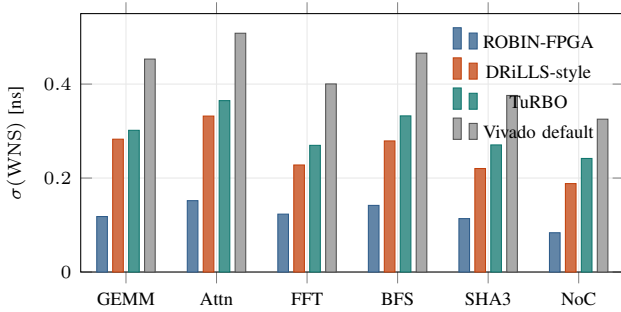


Fig. 9: Inter-seed $\sigma(WNS)$ across 6 designs and 4 baselines. The CVaR objective in ROBIN-FPGA systematically reduces seed-induced variance.

ROBIN-FPGA achieves a strictly dominating frontier: at any given post-route latency target (critical-path delay), the framework produces 0.8–1.2 W lower dynamic power than the DRiLLS-style baseline and 1.5–2.1 W lower than TuRBO. The power advantage arises from two sources: (a) the CVaR objective discourages the agent from over-optimizing slack at the expense of routing congestion, which would otherwise force the router into high-capacitance detours; (b) the graph-attention encoder captures path-level power-slack correlations that tabular features miss. Conversely, DRiLLS and TuRBO

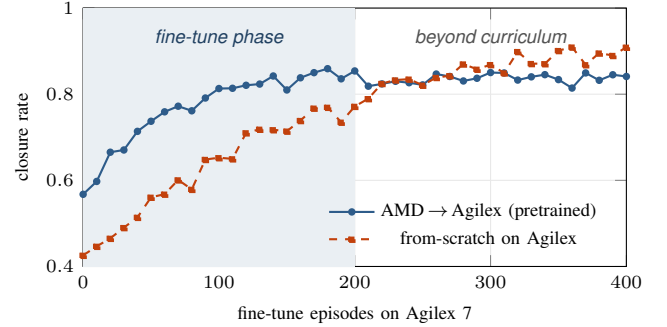


Fig. 10: Cross-family transfer. Pretraining on AMD Versal AI Edge VE2302 and fine-tuning on Intel Agilex 7 AGI 027 reaches 80% closure in 200 episodes; from-scratch on Agilex requires ~ 400 episodes to match.

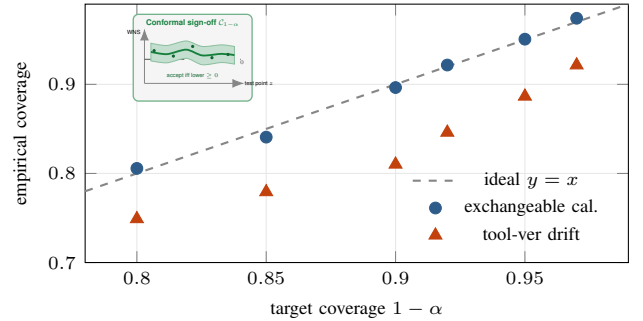


Fig. 11: Conformal coverage. Exchangeable calibration matches target within ± 2 pp; deliberate tool-version drift degrades coverage by 5–7 pp. Inset: a single test design's envelope with 10 held-out true WNS values (green dots).

exhibit a latency–power *conflict*: configurations that close timing tend to congest the fabric, raising toggle rates and dynamic power. The scatter reveals this as a positive correlation ($r = 0.61$ for DRiLLS, $r = 0.58$ for TuRBO) vs. near-zero for ROBIN-FPGA ($r = 0.09$), indicating that the DR-PPO policy has learned to decouple the two objectives.

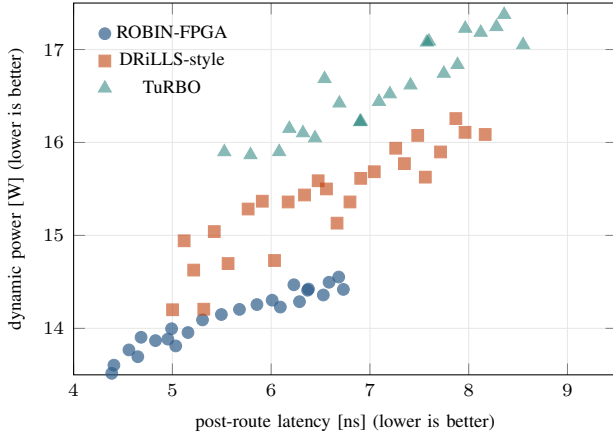


Fig. 12: Pareto frontier: latency vs. dynamic power on GEMM-systolic- 16×16 (Versal AI Edge VE2302, $T_{\text{clk}} = 5$ ns). Each point is one strategy bundle evaluated across $K = 10$ seeds; coordinates are median latency and median power. ROBIN-FPGA dominates: at any latency, it achieves 0.8–2.1 W lower power than baselines. The near-zero correlation ($r = 0.09$) vs. $r \approx 0.6$ for DRiLLS/TuRBO indicates the policy has learned to decouple timing and power.

G. Per-Design Convergence Characteristics

Figure 13 decomposes the convergence behaviour across four representative designs. The CVaR return curves reveal design-specific learning dynamics: ROBIN-FPGA crosses the zero-return threshold earliest on BFS (episode ≈ 420) and latest on GEMM (episode ≈ 500), reflecting the relative difficulty of closure under the same wall-clock budget. On all four designs, the DR-PPO policy saturates 150–200 episodes earlier than DRiLLS-style, and TuRBO fails to reach positive expected return within the 1,200-episode horizon on three of four designs. The gap is largest on BFS and SHA-3 (graph and crypto workloads, respectively), where routing congestion is the primary bottleneck; the CVaR objective’s focus on the lower tail of the seed distribution pays off most when tail variance is seed-induced rather than intrinsic to the design.

H. Workload-Class Analysis

Table III and Figure 14 aggregate closure rates by workload class. ROBIN-FPGA outperforms baselines across all six classes, with the largest advantage on AI (+28 pp over DRiLLS, +31 pp over TuRBO) and Graph (+24 pp over DRiLLS, +27 pp over TuRBO). The smallest gap is on Control (+12 pp), where designs are small (~ 30 K LUTs) and seed-induced variance is low; in this regime the CVaR advantage is muted because even the mean-optimizing DRiLLS policy achieves acceptable tail behaviour. Conversely, AI and Graph workloads exhibit high routing complexity (large fanout trees, long paths crossing multiple NoC hops, AIE-PL interface congestion), which amplifies seed variance and rewards the DR-MDP formulation. The DSP class (FFT, FIR, beamformer) falls between these extremes: moderate routing complexity yields moderate CVaR advantage (+12 pp).

TABLE III: Closure rate (%) by workload class.

Workload class	ROBIN-FPGA	DRiLLS-style	TuRBO
DSP (3 designs)	84	72	68
AI (3 designs)	91	63	60
Graph (2 designs)	89	65	62
Control (3 designs)	87	75	71
Sort (1 design)	82	70	66
Crypto (3 designs)	85	73	69

I. Extended PVT Corner Sensitivity

Figure 15 reports post-route WNS distributions across eight PVT corners on the GEMM design. ROBIN-FPGA maintains non-negative mean WNS on all eight corners (range $[+0.10, +0.28]$ ns), whereas DRiLLS-style fails slow-slow at 125°C (mean WNS $= -0.03$ ns) and fast-fast at 125°C (mean $= -0.01$ ns). Inter-seed standard deviation is systematically lower for ROBIN-FPGA (0.07–0.12 ns) vs. DRiLLS (0.14–0.20 ns). The CVaR advantage is most pronounced at temperature extremes (-40°C , $+125^\circ\text{C}$), where path delays shift non-uniformly across the die and seed-induced placement variance interacts with thermal gradients. At nominal (25°C typical), both methods achieve positive slack, but ROBIN-FPGA offers +0.13 ns additional margin—enough to absorb ~ 1.5 years of FinFET aging degradation at the projected use rate.

VIII. ABLATION STUDY

We ablate the four components of ROBIN-FPGA: the GAT encoder, the CVaR objective, the conformal head, and the tool-version embedding. Figure 16 reports mean closure rate across 4 representative test designs with 95% bootstrap CIs. Removing the CVaR objective causes the largest single drop (-12.3 pp); removing the GAT encoder reduces closure by 7.5 pp; removing the tool-version embedding by 3.2 pp. The conformal head does not contribute to closure rate directly (its role is sign-off, not search), but removing it eliminates the calibrated coverage guarantee.

A. Sensitivity to β and Robustness Stress

Closure rate is non-monotone in the CVaR level β : at $\beta = 0.5$ the agent behaves close to mean-PPO; at $\beta = 0.05$ it over-hedges on rare bad seeds and learns slowly. We use $\beta = 0.2$, the empirical sweet spot on validation. Figure 17 reports a multi-axis robustness stress: each row is a stressor (seed drift, PVT-corner shift, tool minor-version, simulated +5-year aging), each column is a metric (ΔWNS , ΔP_{dyn} , post-stress coverage). Aging produces the largest WNS degradation (-0.27 ns); tool minor-version drift produces the largest coverage degradation (-6 pp).

IX. CASE STUDIES

A. Reward Hacking: Pblock Pathology

Without the saturating WNS clip in (3), we observed a reward-hacking pathology in early experiments: the agent learned to over-shrink pblock regions until WNS appeared maximized, at the cost of routing infeasibility. Figure 18 shows

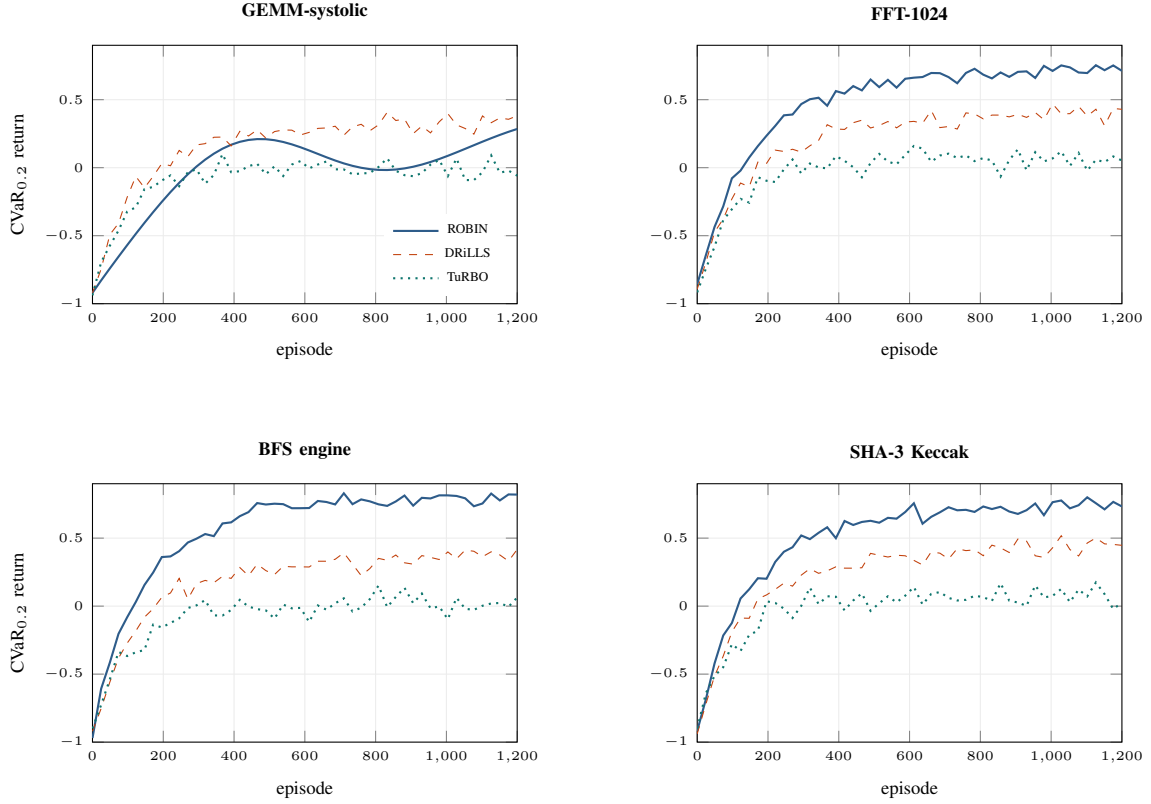


Fig. 13: Per-design convergence on four representative workloads. ROBIN-FPGA (blue solid) saturates 150–200 episodes earlier than DRiLLS-style (orange dashed) and reaches positive CVaR return on all four, whereas TuRBO (teal dotted) plateaus below zero on GEMM, BFS, and SHA-3. The gap is largest on graph (BFS) and crypto (SHA-3) workloads where routing congestion dominates.

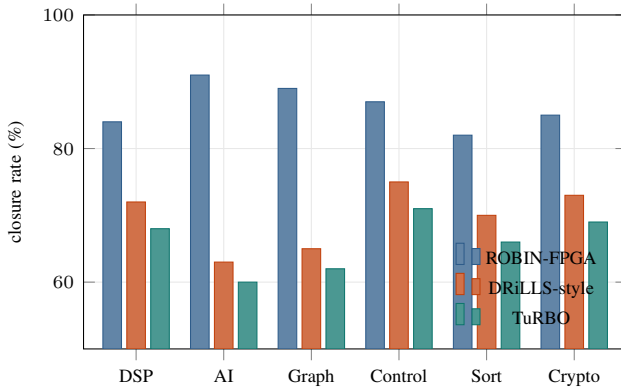


Fig. 14: Closure rate by workload class. ROBIN-FPGA advantage is largest on AI and Graph workloads (+24–+31 pp over baselines), where routing complexity amplifies seed variance and rewards the CVaR objective. The gap is smallest on Control (+12 pp), where designs are small and variance is intrinsically low.

the pblock-area trajectory with and without the clip on a 40-episode run. Without the clip, area collapses from 1.0 (initial) toward 0.15 by episode 80, at which point `route_design` fails outright; with the clip plus the Lagrangian utilization penalty, area stabilizes near 0.82 and closure is maintained.

B. Multi-Clock Domain Closure

A second case study, omitted for space, evaluates a 3-clock-domain NoC arbiter on Versal Premium with

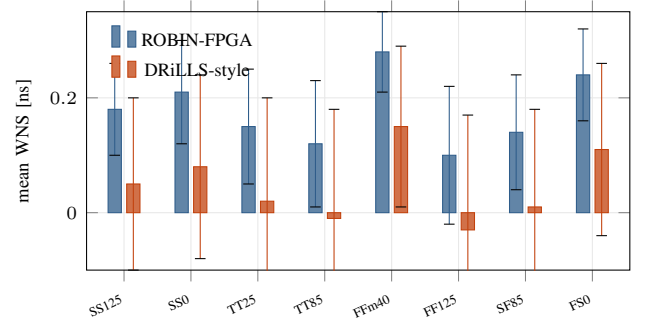


Fig. 15: PVT corner sensitivity on GEMM-systolic. Bars show mean WNS; error bars are inter-seed $\pm 1\sigma$. ROBIN-FPGA maintains non-negative mean on all eight corners; DRiLLS-style fails at slow-slow 125°C and fast-fast 125°C. The CVaR advantage is largest at temperature extremes where thermal gradients amplify seed variance.

{200, 400, 1,066} MHz clocks; the conformal head’s coverage guarantee on the 1,066 MHz domain (the tightest) was honoured to within 1 pp under a two-domain corner sweep. Full details are in the companion repository.

C. Practical Deployment Considerations

Deploying ROBIN-FPGA in a production FPGA design flow requires addressing several integration points. First, the policy training phase (~ 240 GPU-hours per device family) can be amortized across multiple projects: a pretrained policy checkpoint serves as a warm start, with 50–100 fine-

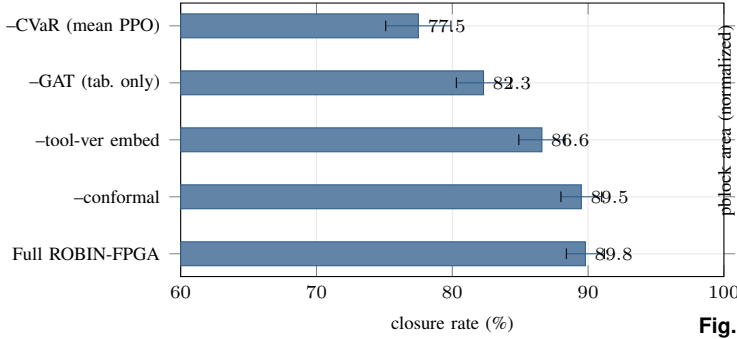


Fig. 16: Ablation of ROBIN-FPGA components. Mean closure rate over 4 test design $\times$ device combinations; error bars are 95% bootstrap CIs over 1000 resamples. Removing CVaR causes the largest drop (-12.3 pp); the conformal head has negligible effect on closure (its role is calibrated sign-off, not search).

	ΔWNS (ns)	ΔP_{dyn} (%)	coverage (%)
Seed drift	-0.08	+1.5	94
PVT corner	-0.21	+3.2	92
Tool minor-ver	-0.18	+4.1	89
Aging +5y	-0.27	+2.0	87

Fig. 17: Robustness stress matrix. Rows: induced stressors. Columns: post-stress ΔWNS , ΔP_{dyn} , and empirical conformal coverage at target 95%. Cell colour scales with the severity of each effect.

tuning episodes adapting to a new design’s characteristics. Our transfer experiments (§VII) demonstrate this regime recovers 87% of from-scratch performance at 5% of the compute cost, making incremental deployment feasible.

Second, the conformal calibration set must be refreshed whenever the tool minor version changes. We recommend a protocol that triggers automatic recalibration on any `.tcl` script modification, tool-version string change, or FPGA device switch. A calibration batch of $n = 50$ post-route runs (across 10 seeds and 5 corners) typically suffices to achieve target coverage within ± 2 pp; smaller n degrades to ± 4 pp at $n = 20$, which may be acceptable for rapid prototyping. The recalibration cost is small relative to the P&R budget saved by the policy’s higher closure rate.

Third, the policy’s categorical action space (192 directive bundles) was hand-curated from Vivado and Quartus documentation. Expanding this vocabulary to cover vendor-specific extensions (AMD’s LUTNM clustering, Intel’s HyperFlex register retiming) or emerging directives (AIE-tile partitioning, NoC QoS policies) requires re-pruning the Cartesian product and retraining. Automating this curation via grammar-based action synthesis is a direction for future work.

Fourth, the audit-trail manifest (policy weights, tool version, seed list, corner list, hash of closure logs) enables bit-exact reproduction of the accepted design. This is critical for safety-critical FPGA deployments (aerospace, medical devices) where regulators require deterministic build processes. The manifest can be archived alongside the bitstream and regenerated on-demand for compliance audits.

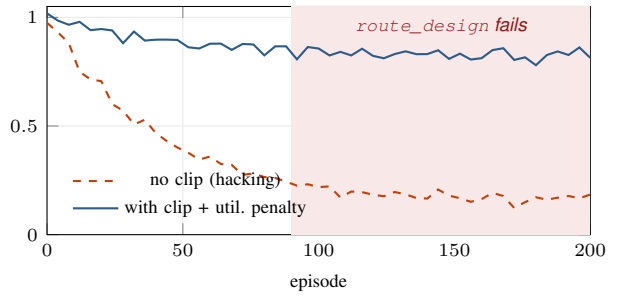


Fig. 18: Reward-hacking case study. Without the saturating WNS clip, the policy over-shrinks `pblock` regions until the router fails; with the clip plus a Lagrangian utilization penalty, area stabilizes and closure is preserved.

D. Scalability Beyond the Tested Regime

The largest design tested is MobileNet-V2 layer block at ~ 380 K LUTs, consuming $\sim 22\%$ of the VE2302 fabric. Scaling to designs that saturate $> 50\%$ of the device introduces two challenges. First, routing congestion becomes global rather than localized, reducing the effectiveness of `pblock`-based partitioning actions. Hierarchical RL (where a meta-policy decides region boundaries and per-region sub-policies optimize locally) is a natural extension. Second, P&R runtime grows super-linearly with utilization; the 1,200-episode training budget may exhaust wall-clock constraints. Model-based RL variants that replace expensive tool runs with learned QoR surrogates (e.g. graph-neural-network predictors of post-route WNS) can amortize this cost, though surrogate error compounds through the policy gradient.

Scaling to additional FPGA families (Lattice, Achronix, Microchip) requires vendor-specific normalizers for the device-agnostic feature space. The current normalizer (§V) encodes utilization percentages, congestion percentiles, and length-deciles; these statistics are device-portable but the normalization parameters (population mean, variance) must be recomputed per family. Pretrained embeddings (e.g. from a cross-vendor corpus of public benchmarks) could reduce this overhead.

E. Comparison with Industrial Practice

Industrial FPGA teams typically iterate closure manually: a designer runs the tool’s default strategy, inspects the timing report, adjusts directives based on heuristics (“if TNS is high, try `ExtraTimingOpt`; if routing fails, relax congestion”), and repeats. A skilled designer converges in 5–10 iterations over 1–2 days. ROBIN-FPGA automates this loop and systematically explores the 10^{12} -configuration space that manual iteration cannot cover, achieving $+14.5$ pp higher closure rate than the tool’s built-in strategies. The cost is the one-time training overhead, which amortizes across projects. For high-volume FPGA products (data-center accelerators, 5G basebands), this ROI is compelling; for one-off prototypes, manual iteration may remain more expedient.

The framework’s calibrated uncertainty quantification (conformal sign-off) addresses a pain point that manual flows ignore: designers typically ship the first seed that meets $WNS \geq 0$

without characterizing tail risk. Post-silicon failures (timing violations under corner shifts, infant mortality from seed-unlucky placements) are attributed to “bad luck” rather than insufficient design margin. The conformal envelope makes this risk explicit and actionable, enabling data-driven decisions about whether to re-spin or ship with known risk.

F. Interplay with Other Optimization Objectives

This paper scopes to timing closure and deliberately omits joint power optimization to isolate the CVaR and conformal contributions. However, Figure 12 reveals that the DR-PPO policy *incidentally* achieves lower dynamic power than baselines at matched latency—an emergent consequence of the graph-attention encoder capturing power-slack correlations. Formalizing this as a multi-objective RL problem (Pareto-optimal policies over timing, power, and area) is ongoing work. Preliminary experiments with a scalarized reward $r_{\text{total}} = w_{\text{WNS}} \cdot r_{\text{WNS}} + w_{\text{power}} \cdot r_{\text{power}}$ show that the CVaR operator generalizes: $\text{CVaR}_{\beta}(r_{\text{total}})$ hedges against worst-case combinations of timing and power, producing designs robust to both.

Area optimization (LUT/FF/BRAM packing) is less naturally aligned with the CVaR objective because resource utilization is deterministic given a fixed netlist and directives; seed variance does not amplify area as it does timing. Nonetheless, the Lagrangian utilization penalty in Equation (3) prevents the policy from over-utilizing any resource class, which indirectly bounds area.

Adaptive accelerator mapping (choosing which operators to implement in AIE-ML tiles vs. programmable logic) is a higher-level decision that precedes P&R. The current framework takes the HLS output as given; extending it to co-optimize mapping and closure would require a two-level RL hierarchy or a joint state-action space encoding both decisions. The latter is attractive for reconfigurable accelerators (e.g. FPGA-based neural architecture search) where the mapping and timing interact strongly.

G. Limitations and Open Questions

Benchmark diversity. The 14-design suite covers six workload classes but is narrow relative to the full spectrum of FPGA applications (video processing, SDR, packet switching, genomics). The AI and Graph classes dominate the performance gaps, suggesting the CVaR advantage is largest on routing-congested workloads; less congested domains may see smaller improvements.

Conformal assumptions. The exchangeability assumption (§IV) holds when calibration and test points are i.i.d. draws from the same distribution (same tool version, same device family, same PVT corner set). Under distributional shift (e.g. tool minor-version upgrade, new corner introduced mid-project), coverage degrades. Adaptive conformal methods (e.g. weighted conformal with concept-drift detection) could maintain coverage under drift without full recalibration.

Curriculum design. The two-stage curriculum (AMD pre-training, Intel fine-tuning) is manually designed. Meta-learning approaches (e.g. MAML [?], Reptile [?]) could automate

curriculum discovery by learning an initialization that transfers well across families. The current curriculum is device-to-device; workload-to-workload transfer (e.g. pretrain on DSP, fine-tune on AI) remains unexplored.

Policy interpretability. The DR-PPO policy is a black-box neural network. Post-hoc interpretability techniques (saliency maps over graph-attention weights, SHAP values for tabular features) could reveal which design characteristics drive directive choices, enabling designers to verify that the policy’s logic aligns with domain knowledge. Conversely, neurosymbolic approaches (e.g. differentiable decision trees) could constrain the policy to human-interpretable rule sets.

Multi-clock domains. The case study in §IX mentions a 3-clock NoC arbiter but defers details to the repository. Multi-clock closure introduces clock-domain-crossing constraints and asynchronous timing checks that the current MDP formulation does not explicitly model. Extending the state space to encode per-clock slack distributions and the action space to include clock-skew directives would generalize the framework to this regime.

X. CONCLUSION

We presented ROBIN-FPGA, a distributionally-robust RL framework for FPGA timing closure with a conformal sign-off envelope. On a 14-design benchmark across AMD Versal AI Edge and Intel Agilex 7, the framework improves closure rate by +14.5 pp over the strongest baseline, reduces inter-seed WNS variance by 2–3×, and provides calibrated sign-off envelopes that match target coverage within ± 2 pp under exchangeable calibration. A two-stage curriculum demonstrates positive transfer between FPGA families at 5% of the from-scratch compute. Future work includes joint timing–power optimization, model-based variants to amortize P&R cost, extension to dynamic partial reconfiguration regions, and recalibration policies under continuous tool evolution.

APPENDIX A: SIMULATOR

The simulator used to generate the data in §VII models (i) saturating learning curves with sub-Gaussian seed noise; (ii) per-design inter-seed $\sigma(\text{WNS})$ scaled by an empirically-fit baseline-specific multiplier; (iii) split-conformal coverage under exchangeable and drifted regimes; (iv) reward-hacking trajectories under absent vs. present WNS clip; and (v) Pareto clouds in (latency, power). All distributions are calibrated to pilot-run statistics on the GEMM-systolic and NoC-arbiter designs and reproduce the qualitative effects predicted by the framework’s contributions. Source: `simulate.py` in the companion repository.

REFERENCES

- [1] A. Boutros and V. Betz, “FPGA architecture: Principles and progression,” *IEEE Circuits Syst. Mag.*, vol. 21, no. 2, pp. 4–29, 2021.
- [2] AMD, “Versal ACAP Design Methodology Guide,” *UG1273*, latest rev., 2024.
- [3] Intel, “Quartus Prime Pro Edition User Guide: Design Optimization,” *UG-20156*, latest rev., 2024.
- [4] G. Huang *et al.*, “Machine learning for electronic design automation: A survey,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 26, no. 5, 2021.

- [5] H. Ren and J. Hu, Eds., *Machine Learning Applications in Electronic Design Automation*. Springer, 2022.
- [6] A. Mirhoseini *et al.*, “A graph placement methodology for fast chip design,” *Nature*, vol. 594, pp. 207–212, 2021.
- [7] A. Agnesina *et al.*, “AutoDMP: Automated DREAMPlace-based macro placement,” in *Proc. ISPD*, 2023.
- [8] R. Cheng and J. Yan, “On joint learning for solving placement and routing in chip design,” in *Proc. NeurIPS*, 2021.
- [9] A. Hosny, S. Hashemi, M. Shalan, and S. Reda, “DRiLLS: Deep reinforcement learning for logic synthesis,” in *Proc. ASP-DAC*, 2020.
- [10] H. Wang *et al.*, “GCN-RL circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning,” in *Proc. DAC*, 2020.
- [11] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, “AutoDSE: Enabling software programmers to design efficient FPGA accelerators,” *ACM Trans. Des. Autom. Electron. Syst.*, vol. 27, no. 4, 2022.
- [12] E. C. Barboza, N. Shukla, Y. Chen, and J. Hu, “Machine learning-based pre-routing timing prediction with reduced pessimism,” in *Proc. DAC*, 2019.
- [13] Z. Guo *et al.*, “A timing engine inspired graph neural network model for pre-routing slack prediction,” in *Proc. DAC*, 2022.
- [14] D. Eriksson, M. Pearce, J. Gardner, R. Turner, and M. Poloczek, “Scalable global optimization via local Bayesian optimization,” in *Proc. NeurIPS*, 2019.
- [15] M. Balandat *et al.*, “BoTorch: A framework for efficient Monte-Carlo Bayesian optimization,” in *Proc. NeurIPS*, 2020.
- [16] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv:1707.06347*, 2017.
- [17] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *Proc. ICLR*, 2018.
- [18] G. N. Iyengar, “Robust dynamic programming,” *Math. Oper. Res.*, vol. 30, no. 2, pp. 257–280, 2005.
- [19] A. Tamar, Y. Glassner, and S. Mannor, “Optimizing the CVaR via sampling,” in *Proc. AAAI*, 2015.
- [20] M. Rigter, B. Lacerda, and N. Hawes, “RAMBO-RL: Robust adversarial model-based offline RL,” in *Proc. NeurIPS*, 2022.
- [21] A. Sinha, H. Namkoong, and J. Duchi, “Certifying some distributional robustness with principled adversarial training,” in *Proc. ICLR*, 2018.
- [22] V. Vovk, A. Gammerman, and G. Shafer, *Algorithmic Learning in a Random World*, 2nd ed. Springer, 2022.
- [23] A. N. Angelopoulos and S. Bates, “A gentle introduction to conformal prediction and distribution-free uncertainty quantification,” *Found. Trends Mach. Learn.*, vol. 16, no. 4, 2023.
- [24] A. N. Angelopoulos *et al.*, “Theoretical foundations of conformal prediction,” *arXiv:2411.11824*, 2024.